

Geometric Interpretation of SVM and Kernel Methods

Introduction to Machine Learning (CSC311)
Spring 2020

University of Toronto

Binary Classification with a Linear Model

- Classification: Predict a discrete-valued target
- Binary classification: Targets $t \in \{-1, +1\}$
- Linear model:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

$$y = \text{sign}(z)$$

- Question: How should we choose $\mathbf{w}$ and b ?

Zero-One Loss

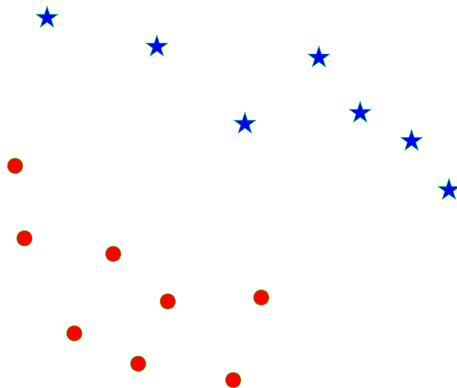
- We can use the 0 – 1 loss function, and find the weights that minimize it over data points

$$\begin{aligned}\mathcal{L}_{0-1}(y, t) &= \begin{cases} 0 & \text{if } y = t \\ 1 & \text{if } y \neq t \end{cases} \\ &= \mathbb{I}\{y \neq t\}.\end{aligned}$$

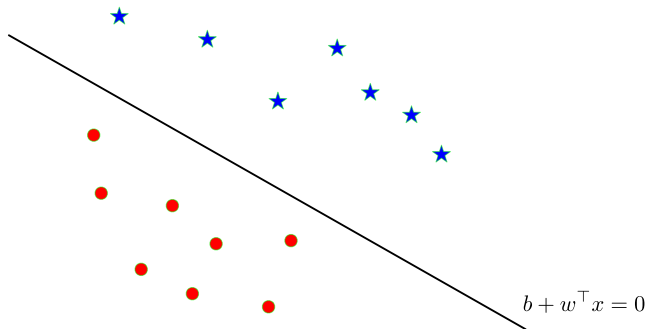
- But minimizing this loss is computationally difficult, and it can't distinguish different hypotheses that achieve the same accuracy.
- We investigated some other loss functions that are easier to minimize, e.g., logistic regression with the cross-entropy loss $\mathcal{L}_{\text{CE}}$.
- Let's consider a different approach, starting from the geometry of binary classifiers.

Separating Hyperplanes

Suppose we are given these data points from two different classes and want to find a linear classifier that separates them.

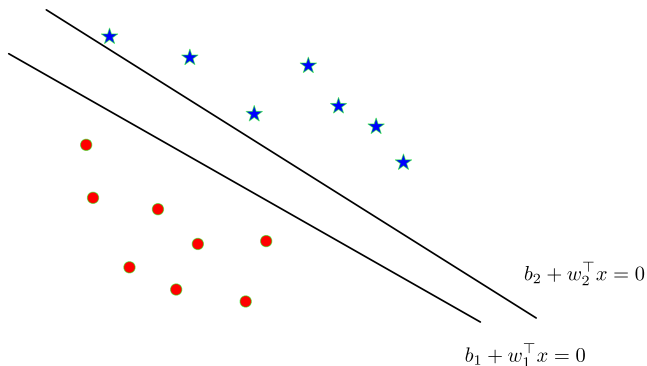


Separating Hyperplanes



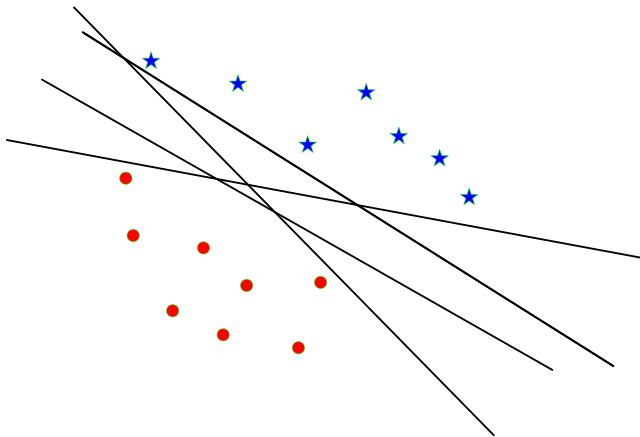
- The decision boundary looks like a line because $\mathbf{x} \in \mathbb{R}^2$, but think about it as a $D - 1$ dimensional hyperplane.
- Recall that a hyperplane is described by points $\mathbf{x} \in \mathbb{R}^D$ such that $f(\mathbf{x}) = \mathbf{w}^T x + b = 0$.

Separating Hyperplanes



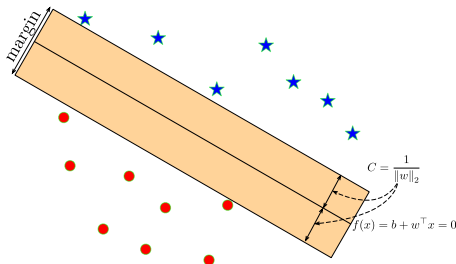
- There are multiple separating hyperplanes, described by different parameters $(\mathbf{w}, b)$.

Separating Hyperplanes



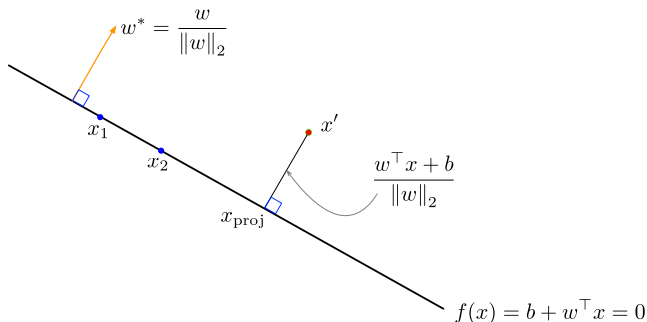
Optimal Separating Hyperplane

Optimal Separating Hyperplane: A hyperplane that separates two classes and maximizes the distance to the closest point from either class, i.e., maximize the **margin** of the classifier.



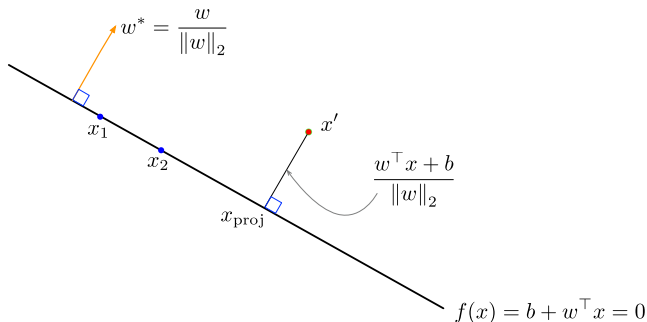
Intuitively, ensuring that a classifier is not too close to any data points leads to better generalization on the test data.

Geometry of Points and Planes



- Recall that the decision hyperplane is orthogonal (perpendicular) to $\mathbf{w}$.
- The vector $\mathbf{w}^* = \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$ is a unit vector pointing in the same direction as $\mathbf{w}$.
- The same hyperplane could equivalently be defined in terms of $\mathbf{w}^*$.

Geometry of Points and Planes



The (signed) distance of a point $\mathbf{x}'$ to the hyperplane is

$$\frac{\mathbf{w}^T \mathbf{x}' + b}{\|\mathbf{w}\|_2}$$

Maximizing Margin as an Optimization Problem

- Recall: the classification for the i -th data point is correct when

$$\text{sign}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = t^{(i)}$$

- This can be rewritten as

$$t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) > 0$$

- Enforcing a margin of C :

$$t^{(i)} \cdot \underbrace{\frac{(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2}}_{\text{signed distance}} \geq C$$

Maximizing Margin as an Optimization Problem

Max-margin objective:

$$\begin{aligned} & \max_{\mathbf{w}, b} C \\ \text{s.t. } & \frac{t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq C \quad i = 1, \dots, N \end{aligned}$$

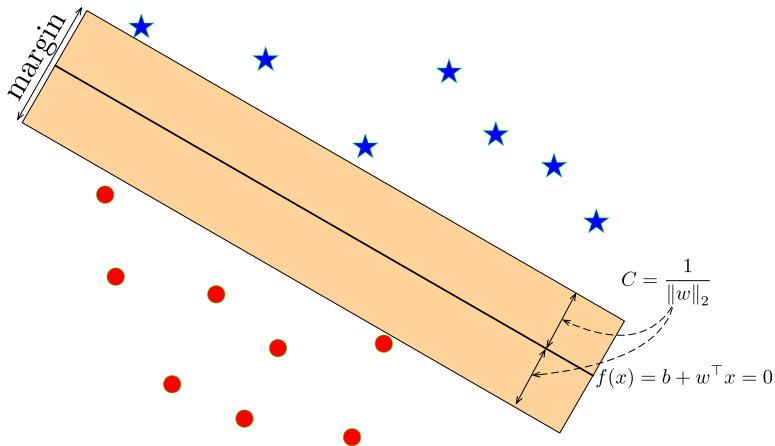
Plug in $C = 1/\|\mathbf{w}\|_2$ and simplify:

$$\underbrace{\frac{t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq \frac{1}{\|\mathbf{w}\|_2}}_{\text{geometric margin constraint}} \iff \underbrace{t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1}_{\text{algebraic margin constraint}}$$

Equivalent optimization objective:

$$\begin{aligned} & \min \|\mathbf{w}\|_2^2 \\ \text{s.t. } & t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad i = 1, \dots, N \end{aligned}$$

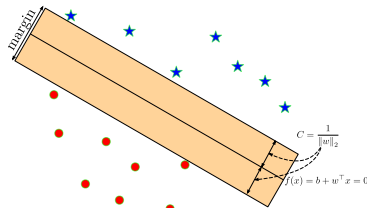
Maximizing Margin as an Optimization Problem



Maximizing Margin as an Optimization Problem

Algebraic max-margin objective:

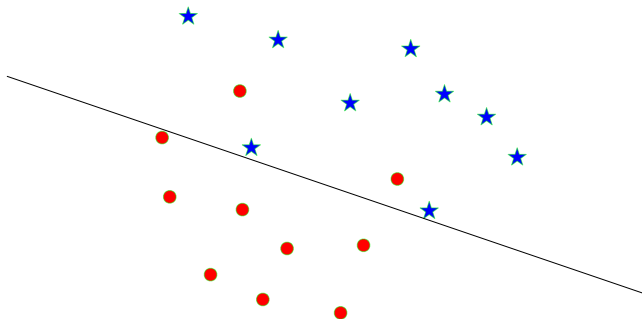
$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad i = 1, \dots, N \end{aligned}$$



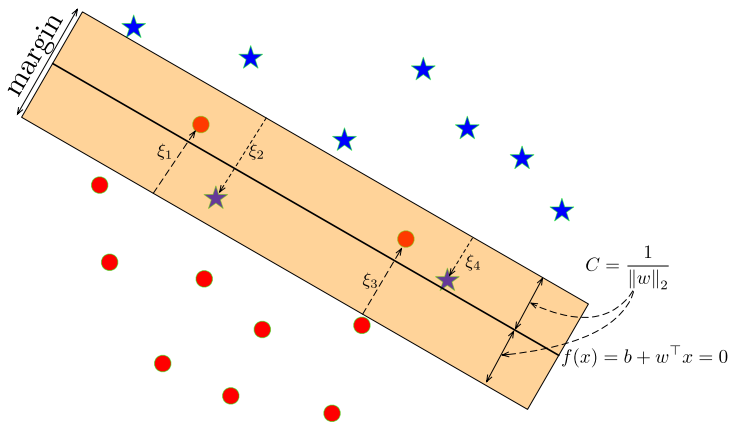
- Observe: if the margin constraint is not tight for $\mathbf{x}^{(i)}$, we could remove it from the training set and the optimal $\mathbf{w}$ would be the same.
- The important training examples are the ones with algebraic margin 1, and are called **support vectors**.
- Hence, this algorithm is called the (hard) **Support Vector Machine (SVM)** (or Support Vector Classifier).
- SVM-like algorithms are often called **max-margin** or **large-margin**.

Non-Separable Data Points

How can we apply the max-margin principle if the data are **not** linearly separable?



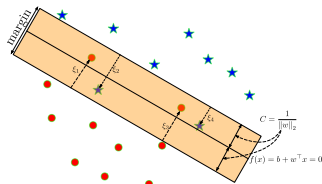
Maximizing Margin for Non-Separable Data Points



Main Idea:

- Allow some points to be within the margin or even be misclassified; we represent this with **slack variables** ξ_i .
- But constrain or penalize the total amount of slack.

Maximizing Margin for Non-Separable Data Points



- Soft margin constraint:

$$\frac{t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq C(1 - \xi_i),$$

for $\xi_i \geq 0$.

- Penalize $\sum_i \xi_i$

Maximizing Margin for Non-Separable Data Points

Soft-margin SVM objective:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \dots, N \\ & \xi_i \geq 0 \quad i = 1, \dots, N \end{aligned}$$

- γ is a hyperparameter that trades off the margin with the amount of slack.
 - ▶ For $\gamma = 0$, we'll get $\mathbf{w} = 0$. (Why?)
 - ▶ As $\gamma \rightarrow \infty$ we get the hard-margin objective.
- Note: it is also possible to constrain $\sum_i \xi_i$ instead of penalizing it.

From Margin Violation to Hinge Loss

Let's simplify the soft margin constraint by eliminating ξ_i . Recall:

$$\begin{aligned} t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) &\geq 1 - \xi_i & i = 1, \dots, N \\ \xi_i &\geq 0 & i = 1, \dots, N \end{aligned}$$

- Rewrite as $\xi_i \geq 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$.
- **Case 1:** $1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \leq 0$
 - ▶ The smallest non-negative ξ_i that satisfies the constraint is $\xi_i = 0$.
- **Case 2:** $1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) > 0$
 - ▶ The smallest ξ_i that satisfies the constraint is $\xi_i = 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$.
- Hence, $\xi_i = \max\{0, 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\}$.
- Therefore, the slack penalty can be written as

$$\sum_{i=1}^N \xi_i = \sum_{i=1}^N \max\{0, 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\}.$$

- We sometimes write $\max\{0, y\} = (y)_+$

From Margin Violation to Hinge Loss

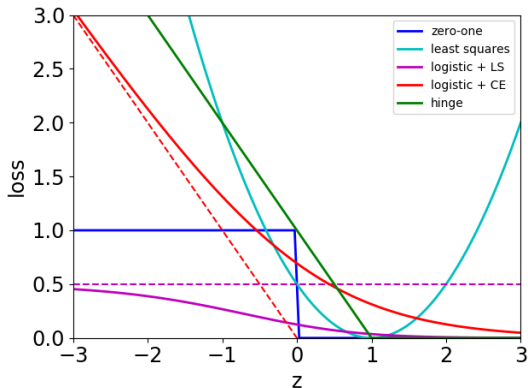
If we write $y^{(i)}(\mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b$, then the optimization problem can be written as

$$\min_{\mathbf{w}, b, \xi} \sum_{i=1}^N \left(1 - t^{(i)} y^{(i)}(\mathbf{w}, b)\right)_+ + \frac{1}{2\gamma} \|\mathbf{w}\|_2^2$$

- The loss function $\mathcal{L}_H(y, t) = (1 - ty)_+$ is called the **hinge** loss.
- The second term is the L_2 -norm of the weights.
- Hence, the soft-margin SVM can be seen as a linear classifier with hinge loss and an L_2 regularizer.

Revisiting Loss Functions for Classification

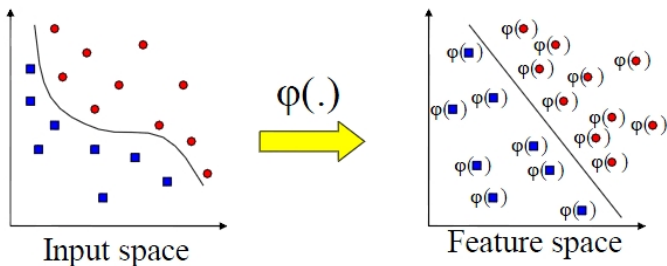
Hinge loss compared with other loss functions



Kernel Methods

Non-Linear Decision Boundaries

- We talk about SVM: Max margin **linear** classifier
- Linear is limiting, how do we get non-linear decision boundaries?
- Feature mapping $\mathbf{x} \mapsto \phi(\mathbf{x})$



- How do we find good features?
- If features are in a high dimension, the computation cost might be large. Can we avoid it?

Motivation of Kernels

- Let's say that we want a quadratic decision boundary
- What feature mapping do we need?
- One possibility (ignore arbitrary $\sqrt{2}$ for now)

$$\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \dots, \sqrt{2}x_d, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \dots, \sqrt{2}x_{d-1}x_d, x_1^2, \dots, x_d^2)$$

Pairwise is over $i < j$

- We have $\dim(\phi(\mathbf{x})) = \mathcal{O}(d^2)$, which is problematic for large d .
- How can this be addressed?

Kernel Trick Idea

- Linear algorithms are based on inner-product.
- What if you could compute the inner product without computing $\phi(\mathbf{x})$?
- Our previous example:

$$\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \dots, \sqrt{2}x_d, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \dots, \sqrt{2}x_{d-1}x_d, x_1^2, \dots, x_d^2)$$

- What is the inner product $\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$ of $\mathbf{x}$ and $\mathbf{y}$?

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = 1 + \sum_{i=1}^d 2x_iy_i + \sum_{i,j=1}^d x_ix_jy_iy_j = (1 + \langle \mathbf{x}, \mathbf{y} \rangle)^2$$

- Let us denote the inner product between mapped features $\phi(x)$ and $\phi(y)$ by $K(\mathbf{x}, \mathbf{y}) \triangleq \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$.
- We can compute K in $O(d)$ memory and compute time.
- K is called the (polynomial) [kernel](#).

Kernelizing SVMs

- Recall that the SVM is defined using the following constrained optimization problem:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \dots, N \\ & \xi_i \geq 0 \quad i = 1, \dots, N \end{aligned}$$

- This is called the primal formulation. We can instead solve a dual optimization problem to obtain $\mathbf{w}$.
- Without going into detail, we have that $\mathbf{w} = \sum_{i=1}^N \alpha_i t^{(i)} \mathbf{x}^{(i)}$ for the vector $(\alpha_i)_{i=1}^N$ being the solution of

$$\begin{aligned} \max_{\alpha_i \geq 0} \quad & \left\{ \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N t^{(i)} t^{(j)} \alpha_i \alpha_j (\mathbf{x}^{(i)\top} \mathbf{x}^{(j)}) \right\} \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C; \quad \sum_{i=1}^N \alpha_i t^{(i)} = 0 \end{aligned}$$

- The prediction of SVM is $y(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$.

Kernelizing SVMs

- If instead of using input $\mathbf{x}$, we first map it to a feature space $\phi(\mathbf{x})$, we get a similar dual optimization problem with the difference that we have to substitute the inner product in the input space $\mathbf{x}^{(i)\top} \mathbf{x}^{(j)}$ with the inner product in the feature space $\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)})$. But this inner product is the kernel between two inputs, i.e., $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$. So we write

$$\max_{\alpha_i \geq 0} \left\{ \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N t^{(i)} t^{(j)} \alpha_i \alpha_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \right\} \quad \text{s.t. } 0 \leq \alpha_i \leq C; \sum_{i=1}^N \alpha_i t^{(i)} = 0$$

- The weight vector would be $\mathbf{w} = \sum_{i=1}^N \alpha_i t^{(i)} \phi(\mathbf{x}^{(i)})$. And the prediction of SVM would be

$$\begin{aligned} y(\mathbf{x}) &= \text{sign}(\mathbf{w}^\top \phi(\mathbf{x})) = \text{sign} \left(\sum_{i=1}^N \alpha_i t^{(i)} \phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}) \right) \\ &= \text{sign} \left(\sum_{i=1}^N \alpha_i t^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}) \right). \end{aligned}$$

- Note that the computation is all expressed in terms of kernels between different points. We do not need to compute the feature mapping $\phi(x)$ explicitly.

- Examples of kernels: **kernels measure similarity**

1. Polynomial

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = (\mathbf{x}^{(i)T} \mathbf{x}^{(j)} + 1)^d,$$

where d is the degree of the polynomial, e.g., $d = 2$ for quadratic

2. Gaussian/RBF

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{2\sigma^2}\right)$$

3. Sigmoid

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \tanh\left(\beta(\mathbf{x}^{(i)T} \mathbf{x}^{(j)}) + a\right)$$

- Kernel functions can be defined for non-vectorized data, e.g., string kernel, graph kernel, etc.
- Each kernel computation corresponds to an inner product in the feature space (and not the input space)
 - ▶ Calculation for particular mapping $\phi(\mathbf{x})$ implicitly maps to high-dimensional space

Kernel Functions

- Mercer's Theorem (1909): any reasonable kernel corresponds to some feature space
- Reasonable means that the Gram matrix is positive semidefinite

$$K_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

- We can build complicated kernels so long as they are positive semidefinite.
- We can combine simple kernels together to make more complicated ones

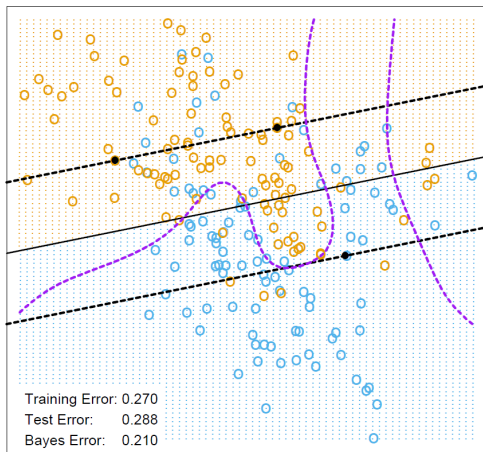
Basic Kernel Properties

- Positive constant function is a kernel: for $\alpha \geq 0$, $K'(x_1, x_2) = \alpha$
- Positively weighted linear combinations of kernels are kernels: if $\forall i, \alpha_i \geq 0$, $K'(x_1, x_2) = \sum_i \alpha_i K_i(x_1, x_2)$
- Products of kernels are kernels: $K'(x_1, x_2) = K_1(x_1, x_2)K_2(x_1, x_2)$
- The above transformations preserve positive semidefinite functions
- We can use kernels as building blocks to construct complicated feature mappings

Kernel Feature Space

- Kernels let us express very large feature spaces
 - ▶ polynomial kernel $(1 + \mathbf{x}^{(i)\top} \mathbf{x}^{(j)})^d$ corresponds to feature space with the dimension exponential in d
 - ▶ Gaussian kernel has infinitely dimensional features
- Linear separators in these super high-dimensional spaces correspond to **highly non-linear decision boundaries** in the input space

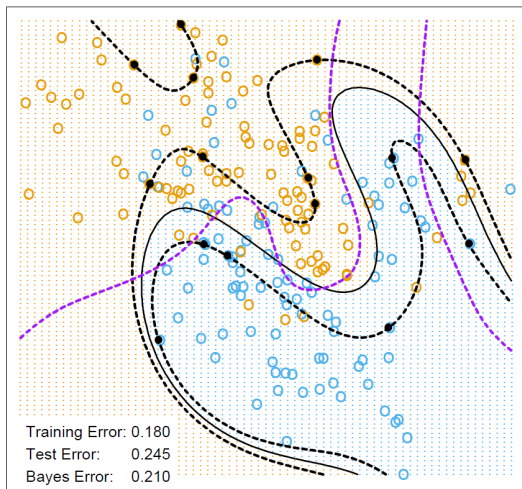
Example: Linear SVM



- Solid line - decision boundary. Dashed - $+1/-1$ margin. Purple - Bayes optimal
- Solid dots - Support vectors on margin

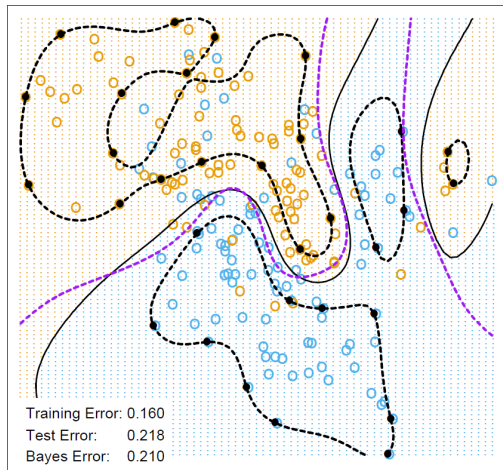
[Image credit: "Elements of statistical learning"]

Example: Degree-4 Polynomial Kernel SVM



[Image credit: "Elements of statistical learning"]

Example: Gaussian Kernel SVM



[Image credit: "Elements of statistical learning"]

Kernel Method Beyond SVMs

- Kernel method is not limited to SVMs.
- When can we apply the kernel trick?

Representer Theorem

If $\mathbf{w}^*$ is defined as

$$\mathbf{w}^* = \arg \min \sum_{i=1}^N L \left(\left\langle \mathbf{w}, \phi(\mathbf{x}^{(i)}) \right\rangle, t^{(i)} \right) + \lambda \|\mathbf{w}\|^2,$$

then $\mathbf{w}^* \in \text{span}\{\phi(x_1), \dots, \phi(x_N)\}$, i.e. $\mathbf{w}^* = \sum_{i=1}^N \alpha_i \phi(x_i)$ for some $\alpha \in \mathbb{R}^N$.

- Proof idea: The subspace that is orthogonal to the span doesn't impact the loss, but increases the norm $\Rightarrow$ We can set the orthogonal part of $\mathbf{w}$ to zero.
- We assume you can predict using inner-product.

Optimization

- We can compute

$$\langle \mathbf{w}, \phi(\mathbf{x}) \rangle = \left\langle \sum_{i=1}^N \alpha_i \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}) \right\rangle = \sum_{i=1}^N \alpha_i \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}) \rangle = \sum_{i=1}^N \alpha_i K(\mathbf{x}^{(i)}, \mathbf{x})$$

- Similarly for the regularizer

$$\begin{aligned} \|\mathbf{w}\|^2 &= \left\langle \sum_{i=1}^N \alpha_i \phi(\mathbf{x}^{(i)}), \sum_{j=1}^N \alpha_j \phi(\mathbf{x}^{(j)}) \right\rangle = \sum_{i,j=1}^N \alpha_i \alpha_j \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle \\ &= \sum_{i,j=1}^N \alpha_i \alpha_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \end{aligned}$$

- We can optimize without computing $\phi(\mathbf{x})$.

$$\alpha = \operatorname{argmin}_{\alpha \in \mathbb{R}^N} \sum_{i=1}^N L \left(\sum_{j=1}^N \alpha_j k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}), t^{(i)} \right) + \lambda \sum_{i,j=1}^N \alpha_i \alpha_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

Other Kernel Methods

- Kernel Logistic regression
 - ▶ We can think of logistic regression as minimizing $\log(1 + \exp(-t^{(i)} \mathbf{w}^T \mathbf{x}^{(i)}))$
 - ▶ If you use L_2 regularization (Gaussian prior) this fits the representer theorem.
 - ▶ Performance is close to SVM
- PCA
 - ▶ A bit trickier to show how to only use kernels.
 - ▶ Equivalent to first using a non-linear transformation to high dimension then use linear projection to low dimension.
- Kernel Bayesian methods (not covered in this course)
 - ▶ Gaussian processes

- The kernel trick is not limited to SVM, but is most common with it.
- Why do the kernel trick and SVM work well together?
- Generalization:
 - ▶ The kernel trick allows us to work in very high dimensions.
 - ▶ Regularization allows us to control the complexity of the high dimensional space.
- Computation:
 - ▶ In general, $\mathbf{w}^*$ is a linear combination of the training data N .
 - ▶ This might become an issue for large N .

Summary

- Advantages:

- ▶ Kernels allow very flexible hypotheses spaces
- ▶ Kernel trick allows us to work in very high (or infinite) dimensional space
- ▶ Soft-margin extension permits mis-classified examples
- ▶ Can usually outperform linear SVMs

- Disadvantages:

- ▶ Must choose kernel parameters
- ▶ Large number of support vector $\Rightarrow$ Computationally expensive to predict new points.